

Sistematización y evaluación empírica del aislamiento por origen del navegador para contenido educativo de autor

— Moodle, WordPress, Omeka S, SCORM, H5P y eXeLearning

Ernesto Serrano Collado · info@ernesto.es

2026-06-14

Índice

Resumen	1
1. Introducción	2
2. Background	3
3. Metodología	3
4. Resultados	5
5. Discusión	8
6. Mitigaciones	8
7. Limitaciones	10
8. Ética y divulgación responsable	10
9. Declaración de conflicto de interés	10
10. Disponibilidad de artefactos	11
11. Conclusiones	11
12. Declaración sobre el uso de IA generativa	11
13. Referencias y estándares	12

Moodle, WordPress, Omeka S, SCORM, H5P y eXeLearning

Ernesto Serrano Collado · Independent Researcher, España · ORCID: 0009-0006-3817-1317

Documento a título personal. Declaración de conflicto de interés: el autor colabora en el proyecto eXeLearning y es autor/mantenedor de varias piezas evaluadas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`, `wp-franer`); el modo seguro descrito en la sección 6.2 es aportación propia. Véase la tabla de procedencia en la sección 9.

Resumen

Los recursos educativos interactivos —SCORM, H5P, paquetes de eXeLearning, páginas HTML— a menudo necesitan JavaScript para funcionar. El problema no es JavaScript: es ejecutar JavaScript no confiable dentro de una sesión autenticada del LMS sin una frontera de aislamiento explícita. Este trabajo es una **sistematización (SoK) y evaluación empírica de seguridad** de nueve formas habituales de publicar contenido en Moodle, WordPress y Omeka S, realizada con el código fuente delante y con una sonda de capacidades inocua ejecutada en laboratorio local. El hallazgo central, organizado en una matriz comparativa con un mismo modelo mental (¿ejecuta JS de autor?, ¿en el mismo origen que la plataforma?, ¿hay aislamiento real?), es que cuando el contenido corre en el mismo origen que la plataforma puede leer el DOM autenticado, los formularios con *token* y aprovechar la sesión; cuando corre aislado (origen opaco, `sandbox` sin `allow-same-origin`, o `srcdoc`), no.

Distinguimos tres estados con precisión: (a) las *versiones estables analizadas* de las integraciones de eXeLearning y el SCORM nativo de Moodle ejecutan el contenido en el mismo origen; (b) las *integraciones de eXeLearning*

mantenidas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`) incorporan un modo seguro de origen opaco activado por defecto que cierra esa exposición; (c) `mod_page`, el SCORM nativo y `mod_exeweb/mod_exescorm` siguen siendo *same-origin* por diseño. H5P es un caso aparte: no ejecuta HTML de autor, sino librerías curadas. Sobre `mod_page` precisamos un punto que la literatura suele confundir: su protección real no es el saneamiento *server-side* (usa `noclean=true`), sino la restricción de capacidad/rol (`mod/page:addinstance`).

La IA generativa actúa como factor de frecuencia —facilita pegar HTML/JS sin revisar—; no la medimos empíricamente, la usamos como motivación y contexto de amenaza. La mitigación efectiva vive en el servidor y en las cabeceras, no en confiar en que el contenido “no encuentre” el *token*. Mostramos, y verificamos en navegador (Chromium y Firefox 146/Gecko), un patrón de *hardening* —origen opaco + puente `postMessage` validado + CSP— que mantiene interactividad y *tracking* sin tratar el contenido como parte de la plataforma. A lo largo del texto separamos el estado externo evaluado (versiones estables, propias y de terceros), la mitigación de aportación propia (el modo seguro, sección 6.2; tabla de procedencia en la sección 9) y las propuestas de trabajo futuro (sección 6.3).

Tesis: el riesgo principal de los recursos educativos interactivos no es JavaScript, sino ejecutar JavaScript **de autor** dentro del mismo origen autenticado del LMS. Un modelo basado en origen opaco, **sandbox** estricto, CSP y puente `postMessage` validado permite mantener la interactividad sin confiar en el contenido como si fuera parte de la plataforma.

Palabras clave: seguridad web; LMS; Moodle; eXeLearning; SCORM; H5P; WordPress; Omeka S; aislamiento de iframe; política de mismo origen; **sandbox**; Content Security Policy; `postMessage`; XSS; contenido generado por IA.

1. Introducción

Hoy es trivial pedirle a un asistente de IA “hazme una actividad interactiva en HTML” y pegar el resultado en un recurso del LMS. Ese HTML puede traer `<script>`, `<iframe>`, `onerror=...`, `fetch()` o librerías de terceros que nadie ha revisado —un riesgo de confianza transitiva medido a gran escala en la web [1] y agravado por librerías desactualizadas con vulnerabilidades conocidas [2]—. A la vez, plantillas de foros, *snippets* de StackOverflow, embeds de redes sociales y *trackers* de analítica se copian y pegan tal cual. Un paquete subido por una persona autora —interna o externa— que se renderiza dentro de la sesión de una persona docente o con rol de administración hereda, si no hay aislamiento, parte de los privilegios de esa sesión. La literatura ya señala el contenido y los recursos como vector de riesgo en sistemas de *e-learning* [3] y en aplicaciones de aprendizaje en línea en general [4]; análisis empíricos recientes de vulnerabilidades en LMS (Moodle, Chamilo, ILIAS) lo corroboran [5]. En el ámbito de los Recursos Educativos Abiertos, CEDEC/INTEF advierte que pegar código generado por IA sin poder explicarlo hace “perder la A de Abierto” y puede ocultar funcionalidad insegura [6].

La distinción clave es entre interactividad legítima y código no confiable: una simulación física o un cuestionario necesitan JS; el riesgo aparece cuando ese JS proviene de una fuente que el LMS trata como de confianza sin haberla verificado.

Encuadramos el trabajo como una **sistematización (SoK) y evaluación empírica de seguridad** —un *estudio de diseño*—, no como un reporte de vulnerabilidades: el valor está en la comparación sistemática entre mecanismos y en el patrón de mitigación, no en hallazgos aislados (la naturaleza del problema se acota más abajo).

Pregunta de investigación. ¿Hasta qué punto las formas habituales de publicar recursos educativos interactivos en Moodle, WordPress y Omeka S aíslan el JavaScript de autor respecto a la sesión autenticada de la plataforma?

Subpreguntas:

- **RQ1.** ¿Qué integraciones ejecutan JavaScript de autor?
- **RQ2.** ¿Cuáles lo ejecutan en el mismo origen que la plataforma?
- **RQ3.** ¿Qué mitigaciones permiten mantener interactividad y *tracking* sin exponer el DOM autenticado?
- **RQ4.** ¿Cómo cambia la IA generativa el *modelo de amenaza operativo* de estos recursos? (factor de plausibilidad y frecuencia, **no medido** empíricamente.)

Contribuciones. (1) Una **sistematización** del riesgo de contenido educativo interactivo en tres ejes —¿ejecuta JS de autor?, ¿en el mismo origen que la plataforma?, ¿hay aislamiento real?— con un criterio de clasificación explícito

(sección 3.4); (2) una evaluación empírica de nueve mecanismos de publicación en Moodle, WordPress y Omeka S, anclada a `archivo:línea` y verificada en laboratorio, con una tabla *afirmación* → *evidencia* (sección 4.1); (3) un conjunto de PoC inocuas y reproducibles (solo booleanos, sin *payloads* reutilizables); (4) un patrón de mitigación —origen opaco + `sandbox` estricto + CSP + puente `postMessage` validado— compatible con la interactividad y el *tracking* SCORM, separado en requisitos de diseño (sección 6.2.1) e implementación de referencia (sección 6.2.2); y (5) una discusión —no una medición— sobre el efecto de la IA generativa y los Recursos Educativos Abiertos (REA).

Naturaleza del hallazgo (no es un 0-day). Conviene situar el tipo de problema. Distinguimos cuatro categorías: **(a) vulnerabilidad** —un fallo concreto y corregible (p. ej. un XSS evadible)—; **(b) riesgo por diseño** —comportamiento esperado y documentado, pero peligroso si el contenido no es de confianza (el *same-origin* de SCORM, `noclean=true` en `mod_page`)—; **(c) mala configuración** —capacidades o roles demasiado amplios—; y **(d) cadena de suministro** —librerías H5P, JS de terceros o contenido generado por IA—. Este artículo **no** reivindica *0-days* de terceros: evalúa **fronteras de confianza** en la publicación de recursos educativos y, en particular, la **ausencia de una frontera de origen** entre el contenido de autor y la sesión autenticada del LMS.

2. Background

Política de mismo origen (SOP). El navegador aísla documentos por *origen* (esquema + host + puerto). Un script solo puede leer el DOM, las cookies o el almacenamiento de documentos de su mismo origen; el acceso entre orígenes distintos lanza `SecurityError`. La SOP es la frontera que decide si el contenido de un recurso puede “ver” la sesión del LMS.

iframe sandbox. El atributo `sandbox` restringe lo que un `iframe` puede hacer; los *tokens* (`allow-scripts`, `allow-same-origin`, `allow-popups`, `allow-forms`, `allow-top-navigation`...) reactivan capacidades concretas. Punto crítico: combinar `allow-scripts` con `allow-same-origin` sobre contenido del propio origen **anula el aislamiento** —el propio navegador advierte de que el contenido puede “escapar”— [7], [8]. Sin `allow-same-origin`, el documento obtiene un **origen opaco** (`null`) y queda aislado del padre. Además, **los flags de `sandbox` se propagan a los iframes anidados**: un reproductor de YouTube embebido dentro de un `iframe` opaco hereda la restricción y pierde su propio origen.

Content Security Policy (CSP). Cabecera que limita de qué orígenes puede el documento cargar scripts, imágenes, marcos o hacer conexiones (`script-src`, `img-src`, `frame-src`, `connect-src`, `frame-ancestors`, `object-src`, `base-uri`, `sandbox`...) [9]. Las *whitelists* de CSP son frágiles; se recomiendan estrategias basadas en *nonce*/strict-dynamic [10], y los análisis longitudinales muestran que **desplegar una CSP efectiva en producción es difícil** [11].

SCORM, H5P, eXeLearning. SCORM define que el contenido (el *SCO*) se comunique con el LMS por un objeto JavaScript (`window.API` en 1.2, `API_1484_11` en 2004) descubierto recorriendo `window.parent` [12], [13], [14]. H5P renderiza *tipos de contenido* (librerías) curados por la administración, no HTML de autor [15]. eXeLearning exporta paquetes con HTML/JS del autor (`.elpx`) que las integraciones embeben en un `iframe`.

3. Metodología

3.1 Plataformas y versiones

Analizamos: `mod_exelearning`, `mod_exeweb`, `mod_exescorm`, SCORM nativo (`mod_scorm`), H5P (`mod_h5pactivity` / `core_h5p`), el recurso *Página* (`mod_page`), `wp-exelearning`, `omeka-s-exelearning`, y `wp-franer` como referencia de aislamiento. Las versiones estables analizadas (las que fijan el “estado actual” de cada plataforma) son: **Moodle 5.0.7** (núcleo 2104c372962) con `mod_exelearning` 2c5473d, `mod_exeweb` 60d24fb, `mod_exescorm` e985f4d y el editor eXeLearning 8101f54e; **WordPress** (vía `wp-env`) con `wp-exelearning`; **Omeka S** (Docker, imagen `erseco/alpine-omeka-s:develop`) con `omeka-s-exelearning`; y `wp-franer` 7fbf694. La matriz completa por `archivo:línea` está en `matriz-seguridad.md`; los anexos por plataforma y navegador en `anexos-tecnicos.md`.

Nota de estado. El **modo seguro de origen opaco** descrito en la sección 6.2 es el **diseño por defecto** de las integraciones mantenidas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`), distinto del comportamiento *same-origin* de las versiones estables que aquí se analiza como “estado actual”. El artículo separa explícitamente ambos.

3.2 Entorno

Instancias **Docker locales y desechables**; curso/página de laboratorio marcados POC-SAFE. Las pruebas vivas en navegador se realizaron con **dos motores**: uno basado en Chromium y **Firefox (Gecko 146)** vía Playwright. Verificamos en vivo `mod_exelearning`, `mod_scorm`, `mod_h5pactivity`, `mod_page`, `wp-exelearning` y `omeka-s-exelearning`; el resto (`mod_exeweb`, `mod_exescorm`, `wp-franer`) se verificó sobre código fuente.

Verificación cross-browser. Para confirmar que el aislamiento de origen opaco es una **garantía del estándar web** y no un comportamiento de un motor concreto, replicamos la comprobación en **Firefox 146 (Gecko)** con un script de Playwright reproducible (`evidencias/firefox-isolation-test.cjs`). (i) En una prueba auto-contenida, un `iframe` con `sandbox con allow-same-origin` lee `window.parent` (origen heredado), mientras que *sin* `allow-same-origin` el acceso lanza `SecurityError` e `isOpaqueOrigin` es `true` —medido **desde dentro** del propio `iframe`—. (ii) Los `embeds` reales en modo seguro de `mod_exelearning` (servido por `tokenpluginfile`), `wp-exelearning` y `omeka-s-exelearning` resultan **opacos** también en Firefox: `contentDocument === null` y `contentWindow` lanza `SecurityError` en las **tres** integraciones. El resultado es **idéntico al de Chromium** (`evidencias/resultados-firefox.json`, `evidencias/resultados-firefox-moodle.json`).

3.3 La sonda (probe.js): definición de cada medida

Las pruebas de concepto comparten una sonda que ejecuta una serie de comprobaciones y **solo** devuelve booleanos y nombres de error *redacted*. Nunca lee valores reales, nunca hace `red`, nunca hace `POST`, nunca invoca mutadores SCORM. Cada medida:

Booleano	Qué detecta (no ejerce)
<code>canRunJavascript</code>	el navegador ejecuta el script del autor
<code>isOpaqueOrigin</code>	el documento corre en origen opaco (<code>null</code>)
<code>sandboxAttr / sandboxAllowsSameOrigin</code>	atributo <code>sandbox</code> efectivo y si concede <code>allow-same-origin</code>
<code>canAccessParent / canReadParentDocument</code>	si <code>window.parent/parent.document</code> son accesibles (mismo origen)
<code>canReadParentCookie</code>	si <code>parent.document.cookie</code> es legible (valor siempre <code>REDACTED</code>)
<code>canFindSesskey</code>	si el <code>sesskey/nonce</code> está presente en el DOM <code>same-origin</code> (valor <code>REDACTED</code>)
<code>canFindCourseEditForms / canFindCourseEditLinks</code>	presencia de formularios/enlaces de edición
<code>canAccessTop / canAttemptTopNavigation</code>	alcanzabilidad de la ventana <code>top</code> (no navega; <code>not_attempted</code>)
<code>canOpenPopups</code>	abre y cierra de inmediato un popup 1×1 (inocuo)
<code>canUsePostMessage / canPostMessageToParent</code>	disponibilidad del canal (no envía nada)
<code>canCallScormApi / scormApiFlavor</code>	si <code>window.API/API_1484_11</code> es alcanzable (no lo invoca)
<code>canUseLocalStorage / canUseSessionStorage</code>	acceso a almacenamiento <code>same-origin</code>
<code>sandboxEscape</code>	siempre false : detectado por diseño, nunca intentado

Cuatro paquetes encapsulan la sonda: `evil.elpx` (eXeLearning), `evil.h5p` (control negativo: H5P la filtra), `evil-scorm.zip` (SCORM 1.2 que **detecta** `window.API`) y `evil-page.html` (sonda *inline* para *Página*). Salida típica *redacted*:

```
{ "canRunJavascript": true, "canAccessParent": false, "canReadParentDocument": false,
  "canReadParentCookie": false, "parentCookieValue": "REDACTED", "canFindSesskey": false,
  "sesskeyValue": "REDACTED", "canCallScormApi": true, "isOpaqueOrigin": true,
  "sandboxEscape": false, "error": "SecurityError" }
```

3.4 Criterio de clasificación de riesgo

Clasificamos cada mecanismo con un criterio explícito sobre **siete dimensiones** observables: (i) ¿ejecuta JS de autor?; (ii) ¿en el mismo origen que la plataforma?; (iii) rol mínimo para *publicar* el contenido; (iv) rol del *visitante* que lo ejecuta; (v) ¿hay un *token* de sesión legible en el DOM *same-origin*?; (vi) ¿se emite CSP restrictiva?; (vii) ¿existe una capacidad mutadora *server-side* alcanzable? A partir de ellas separamos el *impacto máximo demostrado* (verificado en laboratorio) del *impacto inferido* (deducido del modelo del navegador). El resumen divulgativo en tres niveles:

- **Bajo**: no ejecuta JS de autor, o lo ejecuta en un origen aislado (opaco/`srcdoc`) sin acceso al padre.
- **Medio**: ejecuta JS aislado pero con canales residuales (popups, `postMessage`), o filtrado por semántica evadible (XSS documentados).
- **Alto**: ejecuta JS de autor en el mismo origen que el LMS (acceso al DOM/sesión autenticada), o en la ventana `top` sin `sandbox`.

La **matriz de riesgo formal** por plataforma —estas siete dimensiones más el impacto demostrado/inferido— está en `matriz-seguridad.md` (sección 1).

3.5 Límites éticos del método

No robamos cookies ni *tokens*, no exfiltramos nada y no atacamos sistemas externos. **La sonda distribuida (probe.js) no hace red ni POST**: solo **detecta** capacidades y muestra una tabla de booleanos *redacted*. Para **confirmar el impacto** (sección 4.2 y anexos) sí ejecutamos, de forma **autorizada y reversible**, peticiones reales —incluidos POST— **únicamente sobre cuentas propias o de laboratorio**, nunca destructivas, nunca en producción y nunca contra terceros; los cambios (p. ej. el nombre/foto del propio perfil) se revirtieron. No publicamos *payloads* reutilizables. Detalle en la sección 8 (Ética y divulgación responsable).

4. Resultados

4.1 Tabla principal

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	Aislamiento real	Nivel de riesgo
mod_page (Página)	Sí (noclean=true; verificado)	Sí (ventana top, sin iframe)	Ninguno <i>server-side</i> ; solo <i>gated</i> por <code>mod/page:addinstance</code>	Alto si lo edita el profesorado (corre en la sesión de cada visitante)
mod_scorm (core)	Sí	Sí	Ninguno (sin sandbox)	Alto
mod_h5pactivity / core_h5p	Parámetros: No (filtra). Librerías: Sí (preloadedJs, código de confianza)	Sí	Parámetros filtrados por semántica; el JS de las librerías corre <i>same-origin</i> sin sandbox	Bajo en contenido; alto si se pueden instalar librerías (h5p:updatelibraries, gestión/administración)
mod_exelearning	Sí	Configurable (secure=opaco por defecto / legacy=same-origin)	Fuerte en <i>secure</i> (origen opaco + bridge), parcial en <i>legacy</i>	Bajo en <i>secure</i> / medio-alto en <i>legacy</i>
mod_exeweb	Sí	Sí	Ninguno	Alto
mod_exescorm	Sí	Sí	Ninguno	Alto
wp-exelearning	Sí	Configurable (secure=opaco por defecto / legacy=same-origin)	Fuerte en <i>secure</i> (origen opaco), parcial en <i>legacy</i>	Bajo en <i>secure</i> / medio-alto en <i>legacy</i>
omeka-s-exelearning	Sí	Configurable (secure=opaco por defecto / legacy=same-origin)	Fuerte en <i>secure</i> (origen opaco), parcial en <i>legacy</i>	Bajo en <i>secure</i> / medio-alto en <i>legacy</i>
wp-franer (referencia)	Sí, aislado	No (srcdoc opaco) + CSP	Fuerte	Bajo

Lectura rápida (responde a RQ1–RQ2): ejecutar JavaScript de autor no es el problema; el problema es ejecutarlo con el mismo origen que el LMS y sin una frontera explícita.

4.1.1 Afirmación → evidencia

Para que el lector distinga sin ambigüedad qué está verificado en vivo, qué en código y qué queda pendiente:

Afirmación	Tipo de evidencia	Fuente
mod_page ejecuta <script> de autor	Vivo (Moodle 5.0.7 local) + código	§4.2; mod/page/view.php:90–93
SCORM nativo corre <i>same-origin</i> sin sandbox	Vivo + código	§4.3; player.php:279–285
H5P filtra los <i>parámetros</i> (no ejecutan)	Vivo (control negativo)	§4.4; poc/evil.h5p
El preloadedJs de una librería H5P corre <i>same-origin</i>	Código + PoC validada + procedimiento manual	§4.4; resultados-h5p-library.json
El modo seguro aísla (origen opaco)	Vivo en Chromium y Firefox 146/Gecko	§4.5–4.6; resultados-firefox*.json
mod_exeweb / mod_exescorm <i>same-origin</i> sin sandbox	Solo código (inferencia)	matriz §2.2
Autoedición persistente del propio perfil (legacy)	Vivo, autorizado y reversible (cuenta propia)	anexo (Confirmación en vivo)
Safari / WebKit	No verificado (trabajo futuro)	—

4.2 *mod_page* (Página) — la protección es la capacidad, no el saneamiento

Un usuario con la capacidad de crear una Página escribe HTML. Al mostrarse, *mod_page* llama a `format_text(..., noclean=true)` (`mod/page/view.php:90-93`, `lib.php:352`). Creamos una Página con `<script>` y `<img onerror>` y la abrimos: **ambos se ejecutaron**. Con `noclean=true` (y `forceclean=0` por defecto), `format_text` **no** pasa por `purify_html()`; el `<script>` del autor se almacena y se ejecuta para **quien lo visualice (incluido el alumnado)**, en la **ventana principal** y en el **mismo origen** que Moodle.

La protección real, por tanto, **no es el filtrado server-side** —no hay— **sino la capacidad/rol**: solo roles autorizados pueden crear/editar una Página (`mod/page:addinstance`). El flag `$CFG->enabletrusttext` está desactivado por defecto, pero **no** condiciona la ejecución en *mod_page*, porque este usa `noclean`. Corregimos así una formulación habitual: la defensa no es “Moodle filtra `<script>`”, sino “solo una persona docente o gestora puede publicar la Página”.

Impacto en la sesión de una persona estudiante (verificado en código). El script —*same-origin*, ventana top— puede: (a) no leer la cookie de sesión (MoodleSession es `HttpOnly` por defecto), pero sí *aprovechar la sesión* leyendo `M.cfg.sesskey` y, como la página principal de Moodle no emite CSP restrictiva, exfiltrar el `sesskey` y datos del DOM y forjar peticiones autenticadas; (b) modificar el DOM de la vista y cambiar persistentemente su propio perfil (nombre/foto reenviando el formulario de `user/edit.php`, confirmado en un Moodle real); (c) enviar mensajes en su nombre —`core_message_send_instant_messages` es `'ajax' => true` con `moodle/site:sendmessage`, capacidad del rol `user`—, lo que habilita una *propagación dependiente de interacción* (el receptor debe abrir un enlace; el cuerpo del mensaje se sana al mostrarse, así que no se auto-ejecuta).

Límite de autopropagación. Una persona estudiante **no** puede plantar HTML ejecutable: los foros usan `trusttext` (con `enabletrusttext` apagado se sana) y carece de `addinstance`. La propagación **escala** si quien abre la Página es una persona docente o gestora: con sus privilegios el script puede crear más Páginas y llamar a servicios privilegiados (p. ej. `core_user_update_users`, `'ajax' => true`).

Alcance: confinado al origen, pero vector latente. Por la SOP, el script no puede leer cookies/DOM de otras webs de la persona usuaria (banca, correo), ni contraseñas guardadas, ni otras pestañas; el alcance se limita a la sesión del propio LMS. Aun así, disponer de JS arbitrario en el origen autenticado es un punto de apoyo persistente: podría aprovechar una vulnerabilidad futura alcanzable desde ese origen (un servicio sin chequeo de capacidad, un IDOR/CSRF), y su impacto **queda acotado por las capacidades del rol que lo abre** (si lo abre un perfil de administración, ejecuta acciones de administración). Además, no necesita actuar de inmediato: el script puede permanecer latente —inofensivo para el alumnado— y condicionar su carga a quién abre la página (`M.cfg.userId/roles`, elementos del DOM visibles solo a perfiles de gestión, o sondeo de capacidades), de modo que solo se activa ante un rol privilegiado. Es, en el peor caso, un ataque dirigido y paciente, siempre **sujeto a las capacidades del visitante**.

El mismo canal de mensajería (`core_message_send_instant_messages`) permitiría además *inducir* a un perfil de mayor privilegio a abrir el recurso, sujeto a la política de mensajería: por defecto (`messagingallusers=0`) solo a contactos/compañeros de curso, y a cualquiera solo si `messagingallusers` está activado. El impacto, una vez abierto, queda **acotado por las capacidades de ese perfil**: con creación de cursos o gestión podría crear un curso (lo reprodujimos *scrapeando* `course/edit.php`) y matricular alumnado (`enrol_manual_enrol_users`, en los cursos que gestiona); un perfil de administración podría modificar cuentas o configuración del sitio. Es decir, el escalado depende del rol y de la configuración; no es automático.

El origen opaco elimina ese punto de apoyo; *mod_page* no lo tiene.

4.3 *SCORM* nativo (*mod_scorm*)

El SCO recorre `window.parent/window.opener` buscando `window.API` (`mod/scorm/loadSCO.php`). El `iframe` se crea **sin atributo sandbox** (`player.php`) y el contenido se sirve del mismo origen (`pluginfile.php`). SCORM, por diseño, asume mismo origen y acceso al padre; aislarlo lo haría incompatible. Su defensa es **server-side**: `confirm_sesskey()` + capacidad `mod/scorm:savetrack` antes de guardar *tracking*. Riesgo: un SCORM malicioso ejecuta JS con el origen de Moodle (lee DOM y cabalga la sesión); la validación limita *qué acciones del servidor* puede forzar, no la lectura del contexto. Es **el menos aislado en el cliente** de los analizados. Limitación adicional del estándar: como el *tracking* ocurre en el cliente, el alumnado puede falsear `completion/score` con `LMSSetValue`, documentado desde hace años [16]; la integridad de la evaluación digital es un campo propio [17].

4.4 H5P (mod_h5pactivity / core_h5p)

H5P inyecta el contenido en un `iframe about:blank` mediante `contentDocument.write()`; ese documento **hereda el origen del padre** (no es opaco, **sin** atributo `sandbox`: `h5piframe.mustache:32-34`), así que su aislamiento **no** proviene del origen. Lo que controla es **qué** ejecuta, y aquí hay que distinguir dos planos.

Plano de los parámetros (control negativo). El texto de autoría de `content.json` pasa por `H5PContentValidator::validateText` aplica `filter_xss()` con una **lista cerrada de etiquetas** que **no incluye** `<script>` y `_filter_xss_attributes` descarta los atributos `on*` (`h5p.classes.php:4303-4384, :5033-5054`); sin tags, el campo se escapa con `htmlspecialchars`. Lo verificamos: `evil.h5p` inyecta `<script>/<img onerror>` en un campo de texto y H5P los descarta. Un `.h5p` que confíe en los **parámetros** para ejecutar JS es, por tanto, un **control negativo**. Aun así, el saneamiento por listas es **históricamente frágil** —las mutaciones de `innerHTML` (mXSS) y el XSS de cliente evaden filtros que no parsean el DOM [18], [19]—, de modo que la robustez no debe descansar solo en el filtro.

Plano de las librerías (la protección es la capacidad, no el saneamiento). Pero las librerías H5P son **código de confianza por diseño**: su `preloadedJs` se carga como `<script src=pluginfile.php/.../core_h5p/...>` y se ejecuta **same-origin y sin sandbox** en la página de Moodle (`player.php:484-500, h5p.js:391-437`). La documentación de H5P lo dice sin ambigüedad —“*JavaScript files ... are by default and necessity allowed for H5P libraries but not for H5P content*”— y recomienda que “*only trusted users should be given permission to update h5p libraries*” [20]. La única barrera para que el contenido de autor introduzca su **propia** librería es una **capacidad**, no un filtro: instalar una librería nueva desde un `.h5p` subido exige `moodle/h5p:updatelibraries` (arquetipo **manager**, `RISK_XSS`), evaluada **contra quien sube el fichero** (`api.php:403-405, helper.php:210-224, h5p.classes.php:1577-1579`). El profesorado solo tiene `moodle/h5p:deploy` (puede usar librerías ya instaladas, no instalar nuevas); un paquete que requiera una librería ausente se **rechaza en validación**. Construimos `evil-h5p-library.h5p` (la librería H5P.ExePocAlert, cuyo `preloadedJs` muestra un aviso y booleanos de capacidad de solo lectura). Que su `preloadedJs` corre **same-origin y sin sandbox** está **verificado sobre el código fuente** (rutas `archivo:línea` citadas) y el paquete está **validado estructuralmente**; la ejecución en vivo *end-to-end* se documenta como **procedimiento manual reproducible** (subida con rol de gestión → el aviso aparece al ver el contenido), ya que la **automatización headless** del *file picker* de Moodle 5 no resultó fiable y queda como trabajo pendiente de automatizar. Es exactamente el patrón de `mod_page`: la defensa real es la **capacidad/rol**, no el saneamiento ni un `sandbox` —y el riesgo es de **confianza en la administración / cadena de suministro**. Este vector está acreditado en el mundo real: en Chamilo, importar un `.h5p` malicioso llegó a **RCE autenticado** (CVE-2026-30875) [21].

H5P tampoco es inmune por el lado del contenido: historial de XSS evadible —MDL-67110 (ejecución de JS) [22], CVE-2024-43439 (XSS reflejado vía mensaje de error de H5P, que esquivó el filtro de contenido) [23], CVE-2024-3111 (stored XSS vía subida de SVG hasta *backdoor* en el plugin H5P de WordPress) [24]—; y su `postMessage` valida `event.source` y `context==='h5p'` pero **no** `event.origin` y envía con `*` —el tipo de comprobación que [25] halló explotable en 84 sitios populares—.

Veredicto matizado: H5P **no** ejecuta el HTML/JS de los *parámetros* (los filtra: control negativo), pero las **librerías** son código de confianza que corre *same-origin* sin `sandbox`; lo que separa al contenido de autor de ejecutar JavaScript es la capacidad `moodle/h5p:updatelibraries` (gestión/administración), no el saneamiento. Mismo patrón que `mod_page`. Evidencia: `evidencias/resultados-h5p-library.json`; PoC: `poc/evil-h5p-library.h5p`.

4.5 eXeLearning en Moodle (mod_exelearning, mod_exeweb, mod_exescorm)

En las versiones estables, `.elpx` se extrae y se sirve por `pluginfile.php` y se muestra en un `iframe` con `sandbox="allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox"`. Es el **mejor aislado de las tres integraciones eXe en Moodle** (bloquea `allow-top-navigation` y `allow-modals`), pero mantiene `allow-same-origin` por tres dependencias del puente SCORM síncrono (el padre lee `iframe.contentDocument` para mapear `objectid`; *pipwerks* recorre `window.parent`; el *teacher-mode* *hider* inyecta CSS en el `contentDocument`). Con ambos flags sobre contenido del propio origen, el `sandbox` **no aísla de verdad**. `mod_exeweb` muestra el contenido **sin sandbox** y `mod_exescorm` tampoco `sandboxea`: ambos son **más débiles**. Verificamos en vivo (modo `legacy`) que el contenido del `iframe` lee `parent.M.cfg.sesskey` y forja `core_user_update_users` (renombró una cuenta de laboratorio, revertida).

4.6 eXeLearning en WordPress y Omeka S

En las versiones estables, wp-exelearning embebe el paquete con `sandbox="allow-scripts allow-same-origin allow-popups"` → **mismo origen**; la sonda obtuvo `canAccessParent: true`, `canReadParentDocument: true` y localizó enlaces a `/wp-admin/`. Además, su proxy REST `/content/<hash>` tiene `permission_callback => '__return_true'` (lectura no autenticada del contenido por hash). En omeka-s-exelearning, la vista pública del ítem usó `sandbox="allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox"`; la sonda midió `canAccessParent: true`, `canReadParentCookie: true`, `isOpaqueOrigin: false` → **mismo origen** (Omeka sí impone validación CSRF obligatoria). WordPress y Omeka no tienen `sesskey`; usan *nonces* y *tokens* CSRF, pero la lógica es la misma: el riesgo no es que el contenido “vea” el *token*, sino que el servidor acepte una acción por venir con un *token* válido que un script same-origin puede leer del DOM.

4.7 wp-franer (referencia de aislamiento)

Embebe el HTML del autor con `srcdoc` (origen opaco) y `sandbox="allow-scripts allow-forms"` —sin `allow-same-origin` ni `popups`—, inyecta una **CSP restrictiva** (`default-src 'none'; ... connect-src 'none'; form-action 'none'; base-uri 'none'`), valida que `event.source` sea un `iframe` conocido y deja el `fetch` autenticado (X-WP-Nonce) en el padre. Es el patrón de referencia: contenido aislado, control por mensajes validados, credenciales y acciones en el padre, CSP que corta la exfiltración. Lo que no es trasladable sin adaptación: el contrato SCORM de eXeLearning es **síncrono** y `srcdoc` impone límites de tamaño/relativos. (Transparencia: *wp-franer* es una **implementación de referencia del propio autor**; véase sección 9.)

5. Discusión

Implicaciones para docentes. El JS pegado de IA o copiado se ejecuta, en la mayoría de integraciones estables, con el origen del LMS y en la sesión de **cada** visitante. La interactividad legítima no exige confiar el origen al contenido.

Implicaciones para la administración. La superficie crítica es *quién* puede publicar HTML/JS (`mod/page:addinstance`, `moodle/site:trustcontent`) y *con qué aislamiento*. Moodle no emite CSP global por defecto; conviene activar el modo seguro de las integraciones que lo ofrecen y tratar los paquetes externos como no confiables. Conviene recordar que la *ausencia de síntomas no prueba seguridad*: un script malicioso *same-origin* puede permanecer latente y activarse solo cuando lo abre una persona con rol de administración (ataque dirigido y paciente, sección 4.2); la prevalencia del XSS persistente de cliente está documentada empíricamente [26]. El XSS en Moodle es objeto de auditoría continua [27], [28].

Impacto de la IA generativa (RQ4) — motivación, no medición. No medimos empíricamente la IA generativa; la tratamos como un factor que cambia el *modelo de amenaza operativo*: no introduce una vulnerabilidad nueva, pero hace más frecuente el patrón peligroso —contenido HTML/JS plausible, copiado sin revisar, publicado por un rol de confianza—, convirtiendo un riesgo latente en cotidiano. Es una hipótesis de plausibilidad y frecuencia, no un resultado cuantitativo; medirla (p. ej. con un estudio de uso) queda como trabajo futuro. CEDEC/INTEF lo formula para los REA: un recurso con código de IA no revisado puede ser legalmente abierto pero pedagógica y técnicamente cerrado e inseguro [6].

Compatibilidad frente a seguridad. El dilema central del modo seguro: el origen opaco aísla, pero **rompe los embeds de terceros** que necesitan su propio origen (YouTube/Vimeo), porque el `sandbox` se propaga al `iframe` anidado. Resolver esto sin renunciar al aislamiento es el objeto de la sección 6.2–sección 6.3.

6. Mitigaciones

Separamos lo externo analizado (6.1) de la mitigación de aportación propia (6.2 —dividida en *requisitos de diseño*, 6.2.1, e *implementación de referencia* en eXeLearning, 6.2.2—) y de las mitigaciones propuestas (6.3).

6.1 Estado externo: el modelo de confianza en el autor

Moodle, en su núcleo, **confía en el autor** para los embeds: el filtro de medios reconoce proveedores conocidos por *allowlist* de URL (clases `core_media_player_external` para YouTube/Vimeo) y embebe el `iframe` **directamente**, **sin sandbox**; H5P confía en sus librerías curadas; SCORM confía en el paquete subido. Es un modelo razonable cuando la autoría es de confianza (profesorado), pero no aísla contenido no confiable.

6.2 Mitigación: requisitos de diseño e implementación de referencia

6.2.1 Requisitos de diseño (independientes de la implementación) Para servir HTML/JS de autor sin exponer la sesión, una implementación debería cumplir, con independencia de la plataforma:

1. **Origen opaco por defecto.** `sandbox` con `allow-scripts` (y `allow-popups/allow-forms` según necesidad) **sin** `allow-same-origin`; el modo aislado como predeterminado y cualquier modo *same-origin* solo como respaldo, con normalización *fail-safe* al modo seguro.
2. **Una única fuente de verdad** para los *tokens* del `sandbox` (un *helper*), en vez de cadenas duplicadas por plantilla.
3. **Directiva `sandbox` en la CSP de la respuesta** del contenido (no solo en el atributo del `iframe`): el documento conserva el origen opaco aunque se abra fuera del `iframe` (pestaña nueva, popup, URL cruda).
4. **CSP restrictiva:** `connect-src 'self'` (corta la exfiltración), `frame-ancestors 'self' / X-Frame-Options: SAMEORIGIN` (anti-*clickjacking*), `object-src 'none'`, `base-uri 'none'`, `form-action` acotado y `Permissions-Policy` que desactive cámara/micrófono/geolocalización/pago.
5. **Neutralización de los tipos activos del paquete** (SVG/XML) con una CSP sin scripts.
6. **Sin acceso directo padre `iframe`.** Donde haya que cruzar (modo docente, calificación SCORM), resolver en servidor (estado por la `src`) o por `postMessage` validado —identidad de ventana + `nonce` + lista cerrada de acciones— con las credenciales (`sesskey/nonce`) solo en el padre y revalidación *server-side*.
7. **Servir por capacidad de solo lectura:** un *token* que solo lea ficheros (no el `sesskey`) o un *content proxy* con `Content-Type` explícito y protección de *path-traversal*.
8. **Tests de seguridad:** que el `sandbox` esperado esté presente por modo, que el *handler* de mensajes rechace orígenes/fuentes desconocidos y que el modo por defecto sea el aislado.

6.2.2 Implementación de referencia en las integraciones de eXeLearning Las tres integraciones mantenidas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`) instancian R1–R8 como su modo seguro activado por defecto —aportación propia del autor; véase la tabla de procedencia en la sección 9—. Lo verificamos en navegador en las tres (origen opaco confirmado en Chromium y Firefox 146/Gecko; el contenido benigno sigue renderizando). Medición antes/después: en *legacy* el contenido lee `parent.M.cfg.ssesskey` y forja una acción; en *secure* el mismo intento lanza `SecurityError` (origen opaco). En Moodle, R7 se cumple con `tokenpluginfile` (un *token* que solo lee ficheros) y el puente SCORM (R6) transmite solo el *score* por `postMessage` validado, con el `sesskey` exclusivamente en el padre.

Tabla de amenazas (activo → condición → impacto → mitigación):

Activo	Amenaza	Condición	Impacto	Mitigación
Sesión del LMS	JS <i>same-origin</i> aprovecha la sesión	el recurso ejecuta JS de autor en el origen del LMS	acciones autenticadas (según el rol de quien visualiza)	origen opaco (sin <code>allow-same-origin</code>)
DOM autenticado	lectura del padre desde el <code>iframe</code>	<code>allow-same-origin</code> presente	exposición de datos/ <code>nonce</code>	<code>sandbox</code> sin <code>same-origin</code> + directiva <code>sandbox</code> en la respuesta
Tracking SCORM	manipulación cliente del <i>score</i>	API JS accesible <code>same-origin</code>	notas falseadas	puente <code>postMessage</code> validado + revalidación <i>server-side</i>
Token de ficheros	exfiltración del <i>token</i> de solo-lectura	CSP permite <code>img/script-src https:</code>	acceso temporal a ficheros del paquete	perfil de CSP estricta (propuesto, sección 6.3) + TTL corto
Persona usuaria privilegiada	carga latente y dirigida	el JS espera —o induce con un mensaje— a que un perfil de gestión/administración abra el recurso	acotado por las capacidades de ese perfil (creación de cursos, matriculación, etc.)	el origen opaco elimina el punto de apoyo; revisión por rol
Otros visitantes	propagación por mensaje	<code>core_message_send_instant_message</code> (rol <code>user</code>)	mensaje no controlado por el usuario	contenido confinado al origen; revisión por rol

Limitación asumida. El origen opaco es incompatible con `embeds` de terceros que necesitan su propio origen (YouTube/Vimeo): el `sandbox` se propaga al reproductor anidado. Para no degradar la experiencia, el modo seguro renderiza el vídeo mediante un *overlay* mediado por el padre (el contenido pide promover un `iframe`; el padre, fuera del *sandbox*, valida y superpone el reproductor real). Implementación actual: *allowlist* de proveedores con reconstrucción de URL canónica. La generalización a cualquier proveedor se trata en la sección 6.3.

6.3 Mitigaciones propuestas (trabajo futuro)

- **Vídeo de cualquier proveedor sin lista blanca (invariante estructural).** En lugar de mantener una *allowlist* de hosts (YouTube/Vimeo) con reconstrucción por proveedor, promover cualquier `iframe` cuyo `src` sea **https + cross-origin al LMS** (rechazando *same-origin*/subdominio/IP/loopback/userinfo). El argumento de seguridad: un `iframe` **cross-origin** no puede leer el LMS (la SOP protege al padre), exactamente el modelo de confianza que Moodle ya usa para embeber YouTube; el invariante “cross-origin” sustituye a la lista de hosts y admite YouTube, Vimeo, Dailymotion, Mediateca de Madrid y cualquier proveedor **sin enumerarlos** ni crear subdominios. Contrapartida a documentar: el autor podría embeber cualquier contenido cross-origin (riesgo de *phishing*/tracking, no de escape); para despliegues de alta seguridad, una opción de “modo estricto” puede reactivar una lista. Alternativa más conservadora: **oEmbed en servidor** (el LMS pide el HTML de embed al proveedor), más seguro pero limitado a proveedores con oEmbed y con coste de *fetch* en servidor [29].
- **Perfil de CSP estricta opcional.** Cerrar el residual detectado: un script en el `iframe` opaco aún puede exfiltrar el *token* de ficheros vía `` porque `img-src/script-src/media-src` admiten `https:`. Un perfil opcional (admin, por defecto desactivado para no romper imágenes externas/MathJax/CDN) que limite esas directivas a los assets del paquete cierra el canal.
- **Defensas de plataforma complementarias.** Mecanismos *secure-by-default* aplicados por el navegador, como **Trusted Types**, han eliminado el DOM-XSS a escala en grandes bases de código [30]; son complementarios al aislamiento por origen opaco —restringen la *capacidad* peligrosa en el límite de la plataforma, la misma lógica que aplicamos a `mod_page` y a las librerías H5P—.
- **Configurabilidad del helper** de embeds por la administración (paridad entre las tres integraciones).

7. Limitaciones

Entorno local y desechable (no medimos producción); versiones concretas (los resultados se atan a los *commits* de la sección 3.1); sin explotación destructiva (demostramos la cadena, no el abuso); verificado en **dos motores** (Chromium y Firefox 146/Gecko), con **Safari/WebKit no probado** (trabajo futuro); el vector de librería H5P está verificado en código y con PoC validada, con la ejecución *end-to-end* como procedimiento manual (la automatización *headless* del *file picker* de Moodle 5 no fue fiable); `mod_exeweb/mod_exescorm` se infieren del código, no de prueba viva; la IA generativa (RQ4) no se mide. El modelo de amenaza se centra en el aislamiento cliente, no en toda la superficie del LMS. La generalización a otras versiones o configuraciones exige re-verificar contra el código correspondiente.

8. Ética y divulgación responsable

Los comportamientos descritos son, en su mayoría, documentados y por diseño: `mod_page` con `noclean=true`, el *same-origin* de SCORM y el modelo de confianza del filtro de medios y de las librerías H5P no son *0-day* de terceros, sino decisiones de diseño conocidas y *gated* por capacidad. El modo seguro de la sección 6.2 es una mitigación ya integrada en software del propio autor (aportación propia).

Divulgación responsable. No procedía una divulgación coordinada con terceros: (i) los comportamientos evaluados son documentados/por diseño, no defectos reportables, por lo que no se notificaron como vulnerabilidades; (ii) el único vínculo con un fallo de terceros es la cita de CVE-2026-30875 (Chamilo), que ya era público y estaba parcheado *upstream* antes de este trabajo; (iii) no se halló ningún *0-day* de terceros sin parchear. Si en el futuro surgiera tal hallazgo, se coordinaría con los mantenedores antes de difundirlo. Publicamos PoC *redacted* (solo booleanos + errores), sin *payloads* reutilizables ni pasos de abuso, y los cambios reversibles de laboratorio se revirtieron.

9. Declaración de conflicto de interés

El autor es colaborador del proyecto eXeLearning y autor/mantenedor de varias piezas del ecosistema analizado. Este doble rol —analista y desarrollador de parte del objeto de estudio— se declara por transparencia; no invalida los resultados (verificables sobre el código y los artefactos), pero el lector debe conocerlo. Para separarlo sin ambigüedad, la siguiente **tabla de procedencia** distingue qué se evalúa, qué vínculo tiene el autor y con qué evidencia:

Componente	Vínculo del autor	Rol en el estudio	Evidencia
Moodle núcleo (<code>mod_page</code> , <code>mod_scorm</code>)	Ninguno (tercero)	Objeto evaluado	Vivo + código
H5P (<code>core_h5p</code>)	Ninguno (tercero)	Objeto evaluado	Parámetros: vivo · librería: código + PoC validada + manual
SCORM (estándar)	Ninguno (tercero)	Objeto evaluado	Vivo + código
<code>mod_exeweb</code> , <code>mod_exescorm</code>	Ecosistema eXeLearning (sin vínculo declarado)	Objeto evaluado	Solo código (inferencia)
<code>mod_exelearning</code> , <code>wp-exelearning</code> , <code>omeka-s-exelearning</code>	Autor/mantenedor	Objeto evaluado (estable) y mitigación propia (modo seguro, 6.2.2)	legacy : vivo · secure : vivo (Chromium + Firefox 146)
<code>wp-franer</code>	Autor	Implementación de referencia (sección 4.7)	Solo código
Safari / WebKit	—	Cobertura de navegador	Pendiente (trabajo futuro)

Las afirmaciones sobre software propio se sostienen sobre la misma evidencia citable (`archivo:línea`, JSON de evidencia, PoC) que las de terceros; ninguna mitigación propia se evalúa solo por inspección del autor.

10. Disponibilidad de artefactos

Se publica un paquete de artefactos reproducible en <https://github.com/ersec/lms-untrusted-content-security-paper> (texto bajo **CC-BY-4.0**; código y PoC bajo **MIT**): la sonda `probe.js` (15 comprobaciones, *redacted*), los paquetes PoC y su `build.sh` —incluida la librería H5P `evil-h5p-library.h5p` que demuestra la ejecución de `preloadedJs`—, la **matriz** completa por `archivo:línea` (`matriz-seguridad.md`), los **anexos** por plataforma/navegador (`anexos-tecnicos.md`), las evidencias en JSON (`evidencias/`, incluida la verificación cross-browser en Firefox de las tres integraciones y sus scripts de Playwright, y `resultados-h5p-library.json`) y el *script* de generación del documento, junto con una guía de reproducibilidad (`REPRODUCIBILITY.md`), un **Makefile** con los objetivos de construcción y las sumas de verificación de los PDF publicados (`pdf/SHA256SUMS`). Las versiones analizadas se identifican por los *commits* de la sección 3.1. Las PoC no contienen *payloads* reutilizables; los PDF de las fuentes con derechos de autor no se redistribuyen (se enlazan por DOI/URL).

11. Conclusiones

JavaScript no es el enemigo: el contenido educativo interactivo a menudo lo necesita. El riesgo nace de ejecutar JavaScript no confiable con el mismo origen que el LMS (RQ1–RQ2): de lo analizado, H5P es el más controlado por diseño (no ejecuta HTML de autor), `mod_page` está protegido por capacidad/rol (no por saneamiento), y SCORM/eXeLearning estable corren *same-origin* por compatibilidad. La respuesta a RQ3 es un patrón concreto y verificado —origen opaco + **sandbox** estricto + CSP (incl. directiva en la respuesta) + puente `postMessage` validado— que mantiene interactividad y *tracking* sin exponer el DOM autenticado; ese patrón es ya el modo por defecto de las integraciones mantenidas de eXeLearning. Sobre RQ4, la IA generativa no crea el fallo y no la medimos: la planteamos como factor de frecuencia que vuelve cotidiano el patrón. Mantenemos separados el estado externo evaluado, la mitigación de aportación propia (secciones 6.2 y 9) y el trabajo futuro (sección 6.3), para que el lector distinga la evaluación del parche propio. La regla que lo resume: **aislar, validar y no confiar en el JavaScript del recurso como si fuera parte de la plataforma**.

*Anexos técnicos (matriz completa por `archivo:línea`, PoC *redacted*, resultados por plataforma/navegador, limitaciones metodológicas): ver `anexos-tecnicos.md` y `matriz-seguridad.md`.*

12. Declaración sobre el uso de IA generativa

En la preparación de este trabajo se utilizaron herramientas de IA generativa (asistentes de redacción y de codificación basados en modelos de lenguaje) como apoyo en la redacción y reestructuración del texto, en la generación y refactorización de las pruebas de concepto y los *scripts* de evidencia, y en la elaboración de tablas. La IA **no figura como autora**. El autor diseñó la investigación, definió la metodología, ejecutó y verificó todas las pruebas en el entorno de laboratorio, comprobó manualmente cada afirmación técnica, el código y las evidencias citadas, y asume la **responsabilidad plena** del contenido.

13. Referencias y estándares

- [1] N. Nikiforakis *et al.*, “You are what you include: Large-scale evaluation of remote JavaScript inclusions,” in *Proc. 2012 ACM conference on computer and communications security (CCS '12)*, 2012, pp. 736–747. doi: [10.1145/2382196.2382274](https://doi.org/10.1145/2382196.2382274).
- [2] T. Lauinger, A. Chaabane, S. Arshad, W. Robertson, C. Wilson, and E. Kirda, “Thou shalt not depend on me: Analysing the use of outdated JavaScript libraries on the web,” in *Proc. 2017 network and distributed system security symposium (NDSS '17)*, 2017. doi: [10.14722/ndss.2017.23414](https://doi.org/10.14722/ndss.2017.23414).
- [3] A. Khamparia and B. Pandey, “Threat driven modeling framework using petri nets for e-learning system,” *SpringerPlus*, vol. 5, no. 1, p. 446, 2016, doi: [10.1186/s40064-016-2101-0](https://doi.org/10.1186/s40064-016-2101-0).
- [4] A. J. J. Joseph and M. Mariappan, “Approaches to overcome security risks and threats in online learning applications,” in *Secure data management for online learning applications*, CRC Press, 2023. doi: [10.1201/9781003264538-3](https://doi.org/10.1201/9781003264538-3).
- [5] S. A.-L. Akacha and A. I. Awad, “Enhancing security and sustainability of e-learning software systems: A comprehensive vulnerability analysis and recommendations for stakeholders,” *Sustainability*, vol. 15, no. 19, p. 14132, 2023, doi: [10.3390/su151914132](https://doi.org/10.3390/su151914132).
- [6] CEDEC (Centro Nacional de Desarrollo Curricular en Sistemas no Propietarios), “Mantener la «a» de abierto en los REA en tiempos de IA.” Accessed: June 14, 2026. [Online]. Available: <https://cedec.intef.es/mantener-la-a-de-abierto-en-los-rea-en-tiempos-de-ia/>
- [7] MDN Web Docs, “The inline frame element: The sandbox attribute.” Accessed: June 13, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe#sandbox>
- [8] WHATWG, “HTML living standard — sandboxing.” Accessed: June 13, 2026. [Online]. Available: <https://html.spec.whatwg.org/multipage/browsers.html#sandboxing>
- [9] S. Stamm, B. Sterne, and G. Markham, “Reining in the web with content security policy,” in *Proceedings of the 19th international conference on world wide web (WWW)*, 2010, pp. 921–930. doi: [10.1145/1772690.1772784](https://doi.org/10.1145/1772690.1772784).
- [10] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, “CSP is dead, long live CSP! On the insecurity of whitelists and the future of content security policy,” in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security (CCS)*, 2016, pp. 1376–1387. doi: [10.1145/2976749.2978363](https://doi.org/10.1145/2976749.2978363).
- [11] S. Roth, T. Barron, S. Calzavara, N. Nikiforakis, and B. Stock, “Complex security policy? A longitudinal analysis of deployed content security policies,” in *Proc. 2020 network and distributed system security symposium (NDSS '20)*, 2020. doi: [10.14722/ndss.2020.23046](https://doi.org/10.14722/ndss.2020.23046).
- [12] Advanced Distributed Learning (ADL), “SCORM 1.2 run-time environment,” Advanced Distributed Learning Initiative, 2001. Accessed: June 13, 2026. [Online]. Available: <https://adlnet.gov/projects/scorm/>
- [13] Advanced Distributed Learning (ADL), “SCORM 2004 4th edition — run-time environment (API API_1484_11),” Advanced Distributed Learning Initiative, 2009. Accessed: June 13, 2026. [Online]. Available: <https://adlnet.gov/projects/scorm/>
- [14] P. Hutchison, “SCORM API wrapper for JavaScript (pipwerks).” Accessed: June 13, 2026. [Online]. Available: <https://github.com/pipwerks/scorm-api-wrapper>
- [15] H5P Group, “H5P documentation — content types and libraries.” Accessed: June 13, 2026. [Online]. Available: <https://h5p.org/documentation>
- [16] P. Hutchison, “Cheating in SCORM.” Accessed: June 13, 2026. [Online]. Available: <https://pipwerks.com/2009/03/22/cheating-in-scorm/>
- [17] P. Dawson, *Defending assessment security in a digital world: Preventing e-cheating and supporting academic integrity in higher education*. Routledge, 2020. doi: [10.4324/9780429324178](https://doi.org/10.4324/9780429324178).
- [18] M. Heiderich, J. Schwenk, T. Frosch, J. Magazinius, and E. Z. Yang, “mXSS attacks: Attacking well-secured web-applications by using innerHTML mutations,” in *Proc. 2013 ACM SIGSAC conference on computer and communications security (CCS '13)*, 2013, pp. 777–788. doi: [10.1145/2508859.2516723](https://doi.org/10.1145/2508859.2516723).
- [19] B. Stock, S. Lekies, T. Mueller, P. Spiegel, and M. Johns, “Precise client-side protection against DOM-based cross-site scripting,” in *Proc. 23rd USENIX security symposium (USENIX security '14)*, 2014, pp. 655–670. Available: <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-stock.pdf>
- [20] H5P, “Security (H5P documentation): Libraries are trusted code.” Accessed: June 14, 2026. [Online]. Available: <https://h5p.org/documentation/installation/security>

- [21] MITRE CVE, “CVE-2026-30875: Authenticated remote code execution in chamilo LMS via H5P package import.” Accessed: June 14, 2026. [Online]. Available: <https://github.com/chamilo/chamilo-lms/security/advisories/GHSA-mj4f-8fw2-hrfm>
- [22] Moodle Tracker, “MDL-67110: XSS in malicious seemingly H5P content.” Accessed: June 13, 2026. [Online]. Available: <https://tracker.moodle.org/browse/MDL-67110>
- [23] Deutsche Telekom Security and Moodle, “CVE-2024-43439: Reflected XSS in Moodle via H5P error message.” Accessed: June 13, 2026. [Online]. Available: <https://github.security.telekom.com/2024/08/reflected-xss-moodle.html>
- [24] CleanTalk PSC, “CVE-2024-3111: Interactive content – H5P (WordPress) stored XSS to backdoor creation.” Accessed: June 13, 2026. [Online]. Available: <https://research.cleantalk.org/cve-2024-3111/>
- [25] S. Son and V. Shmatikov, “The postman always rings twice: Attacking and defending postMessage in HTML5 websites,” in *Proceedings of the network and distributed system security symposium (NDSS)*, 2013. Available: <https://www.ndss-symposium.org/ndss2013/ndss-2013-programme/postman-always-rings-twice-attacking-and-defending-postmessage-html5-websites/>
- [26] M. Steffens, C. Rossow, M. Johns, and B. Stock, “Don’t trust the locals: Investigating the prevalence of persistent client-side cross-site scripting in the wild,” in *Proc. 2019 network and distributed system security symposium (NDSS ’19)*, 2019. doi: 10.14722/ndss.2019.23009.
- [27] R. R. Al-azaiza and T. S. M. Barhoom, “Enhance MOODLE security against XSS vulnerabilities,” *International Journal of Computing and Digital Systems*, vol. 5, no. 5, pp. 421–430, 2016, doi: 10.12785/ijcds/050507.
- [28] Sonar, “Playing dominos with Moodle’s security.” Accessed: June 13, 2026. [Online]. Available: <https://www.sonarsource.com/blog/playing-dominos-with-moodles-security-2/>
- [29] oEmbed, “oEmbed — an open format for embedding content via a provider API.” <https://oembed.com/>.
- [30] P. Wang, B. Á. Gumundsson, and K. Kotowicz, “Adopting trusted types in production web frameworks to prevent DOM-based cross-site scripting: A case study,” in *2021 IEEE european symposium on security and privacy workshops (EuroS&PW)*, 2021, pp. 60–73. doi: 10.1109/EuroSPW54576.2021.00013.